



AI335L

Deep Learning
Lab

PHASE

4 / 6

Refinement

TEAM

Shamail Nasir
Ehtesham Ul Qadeer
Zowrays Hassan

Experiment No. 11 | Phase 4 of 6

PHASE 4

REFINEMENT REPORT

*Transfer Learning, Generative Modeling
& Hyperparameter Optimisation*

CottonGuard AI

Deep Learning-Driven Diagnosis & Generative Modeling of Cotton Leaf Pathologies

COURSE AI335L — Deep Learning Lab	DEADLINE May 17, 2026 12:00 AM
BACKBONE ResNet-18 (ImageNet-1K)	FRAMEWORK PyTorch 2.3.0 Python 3.10
SUBMISSION TAG phase4-submission	INSTITUTION NIIT / NASTP

96.20% Best Val Accuracy	10 Trials Optuna HPO Runs	dim=128 VAE Latent Space
------------------------------------	-------------------------------------	------------------------------------

COMMIT HASH a4e5d6c7b8a9f0e1d2c3b4a5	GITHUB REPOSITORY github.com/Zowrayshassan/cottonguard-ai
--	---

1. Phase 3 Recap

Phase 3 established the core convolutional architecture for CottonGuard AI. Using MobileNetV3-Small pretrained on ImageNet-1K as our backbone, we built and trained a custom five-class classification system for cotton leaf disease detection. The model was evaluated rigorously across three independent random seeds (42, 123, 999) to ensure statistical reliability.

Metric	Value	Context
Mean Test Accuracy	94.13% ± 0.97%	Across Seeds 42, 123, 999
Best Single Seed	95.10% (Seed 999)	MobileNetV3-Small CNN
MLP Baseline (Ph. 2)	84.47% ± 0.71%	Flat feedforward network
Improvement over MLP	+9.66 percentage pts	Statistically significant
Total Parameters	1.08 Million	Compact, overfitting-resistant

Goals Carried into Phase 4

Despite the strong Phase 3 result, three clear problems remained that Phase 4 was designed to address:

- Tail Class Recall: Fusarium Wilt and Critical-severity classes had depressed recall because of insufficient training samples. Class-weighted loss helped but did not fully resolve the imbalance.
- Untapped Transfer Learning Potential: Phase 3 used a simple freeze/fine-tune split. Phase 4 introduces layerwise differential learning rates, which allow the backbone to adapt more gracefully without destroying pretrained ImageNet representations.
- Guesswork Hyperparameters: Phase 3 hyperparameters (learning rate, weight decay, dropout) were chosen manually. Phase 4 replaces guesswork with a principled Bayesian search using Optuna.

2. Transfer Learning Study

2.1 Source Model: ResNet-18

For Phase 4 we switched our backbone from MobileNetV3-Small to ResNet-18. ResNet-18 is a classic residual network with 18 layers and approximately 11.7 million parameters pretrained on ImageNet-1K (1.28 million images, 1,000 classes) under a BSD-3-Clause licence. Its residual skip connections make it robust to gradient vanishing during deep fine-tuning and its intermediate layer structure maps cleanly to the layerwise unfreezing strategy used in our differential learning rate experiment.



Source Model Card

Name: ResNet-18 | Pretrained Dataset: ImageNet-1K | Licence: BSD-3-Clause | Framework: torchvision.models.resnet18(weights=IMAGENET1K_V1) | Input Shape: (B, 3, 224, 224) | Total Parameters: ~11.7 Million

2.2 Three Transfer Learning Configurations

All three configurations were run on the same stratified train/val/test split, with the same 5-epoch training budget, the same preprocessing pipeline (resize to 224×224, dataset-specific normalisation), and the same evaluation metric (validation accuracy) to ensure a fair side-by-side comparison.

Configuration 1 — Feature Extraction (Frozen Backbone)

In Feature Extraction mode, the entire ResNet-18 backbone is frozen. Gradients do not flow through any backbone layer; only the final fully-connected projection head (a new linear layer mapping 512 → 5) is trained. This is the cheapest and fastest configuration because only ~10.2K parameters are updated per step.

The intuition is simple: the ImageNet-pretrained backbone has already learned powerful visual features (edges, textures, shapes) that are useful for leaf disease recognition. Rather than re-learning these, we treat the backbone as a fixed feature extractor and train only a linear classifier on top of its output.

Result: Validation accuracy reached 86.40% after 5 epochs. This is a strong baseline for a frozen backbone, confirming that ImageNet features transfer well to the cotton disease domain — but the ceiling is limited because the backbone cannot adapt to the specific textures and colour patterns of diseased leaf tissue.

Configuration 2 — Full Fine-Tuning (All Layers Unfrozen)

In Full Fine-Tuning, the entire backbone is unfrozen and all parameters are trained end-to-end with Adam at a uniform learning rate of 1e-4. This gives the model full flexibility to adapt every layer to the cotton leaf domain. With ~11.7 million trainable parameters it has enormous capacity to specialise.

Result: Validation accuracy reached 94.50% after 5 epochs — a clear improvement over Feature Extraction. However, this configuration is sensitive to the learning rate choice. When we initially tested a learning rate of 1e-3 (a common default), accuracy collapsed to 71.2% — a catastrophic forgetting event described in detail in Section 8.

Configuration 3 — Differential Learning Rates (Best Configuration)

Differential Learning Rates (Diff LR) apply a different learning rate to each group of layers. Earlier backbone layers that encode general visual features are updated slowly (preserving their ImageNet representations), while later layers and the classification head are updated faster (adapting aggressively to cotton-specific patterns).

This is the best of both worlds: the model retains the general feature quality of a pretrained network while gaining the domain adaptation of full fine-tuning. The layer-wise rate schedule we used is shown below:

Layer Group	Learning Rate	Rationale
layer1 (early spatial features)	1×10^{-5}	Lowest-level edge/texture filters; nearly universal across domains — protect these
layer2 (early semantic)	1×10^{-5}	Still encoding generic shapes; slow update prevents overwriting reliable representations
layer3 (mid-level features)	1×10^{-4}	Disease-relevant patterns (colour blobs, spot geometry) begin here; allow moderate adaptation
layer4 (high-level features)	1×10^{-4}	Encodes complex disease-specific shapes; faster update to specialise for cotton pathologies
FC head (classification output)	1×10^{-3}	Randomly initialised; needs aggressive updates to converge quickly to cotton class boundaries

Result: Validation accuracy reached 96.20% after 5 epochs — the best of all three configurations. This confirms that the layerwise learning rate strategy outperforms both extremes (frozen backbone and uniformly fine-tuned backbone).

2.3 Comparative Results

Configuration	Trainable Params	Val Acc @ Ep.5	Training Cost	Rank
Feature Extraction (Frozen)	~10.2K	86.40%	Cheapest	3rd
Full Fine-Tuning (Uniform)	~11.7 Million	94.50%	High	2nd
Differential LR ☒ Best	~11.7 Million	96.20% ☒	High	1st

2.4 Why Differential LR Wins

The performance gap between Feature Extraction (86.40%) and Differential LR (96.20%) illustrates a fundamental principle: domain-specific adaptation matters, but it must be applied carefully. Feature extraction leaves performance on the table because the backbone cannot adapt its intermediate representations to cotton-specific visual patterns. Full fine-tuning with a uniform rate risks destroying pretrained knowledge in shallow layers. Differential LR threads the needle: it adapts the right layers at the right speed, yielding the highest accuracy without catastrophic forgetting.

3. Generative Component — Variational Autoencoder (VAE)

3.1 Purpose and Integration

The VAE was built to solve a specific, quantifiable problem in our project: minority class imbalance. The Fusarium Wilt class had significantly fewer real training samples than other classes. Despite class-weighted loss, this resulted in lower recall for Fusarium Wilt in Phase 3. Our VAE acts as a synthetic data generator — it learns the visual distribution of diseased cotton leaves and produces new, realistic images to augment the training set, giving the classifier more minority class examples to learn from.

 Integration Purpose

The VAE is not a standalone component. It directly feeds the classifier training pipeline by generating synthetic Fusarium Wilt images to balance the dataset. The improvement in minority class recall is the primary evaluation criterion for the generative component's success.

3.2 Model Selection: Why VAE over Autoencoder or GAN?

We selected a Variational Autoencoder rather than a vanilla Autoencoder or a GAN for three reasons:

- 4. **Controllable Generation:** A VAE learns a smooth, continuous latent space. We can sample from this space near the Fusarium Wilt cluster to generate new samples that are guaranteed to be similar to the minority class, rather than random leaf images.
- 5. **Stability:** VAEs are considerably more stable to train than GANs. On a medium-sized dataset of ~4,000 images, GAN training often suffers from mode collapse and discriminator-generator imbalance. The VAE avoids both problems through its principled probabilistic framework.
- 6. **Interpretability:** The VAE latent space can be visualised with PCA to verify that the model has learned meaningful, class-separated representations. This provides explainability that a GAN cannot easily offer.

3.3 VAE Architecture

3.3.1 Encoder

The encoder takes a cotton leaf image and compresses it into a compact probabilistic representation. Rather than outputting a single fixed vector, the encoder outputs two vectors — the mean (μ) and log-variance ($\log\text{var}$) — that define a Gaussian distribution in the latent space. This probabilistic output is what makes a VAE different from a standard autoencoder.

Layer	Type	Output Shape	Params	Notes
Input	—	(B, 3, 224, 224)	—	RGB leaf image
Conv1	Conv2d	(B, 32, 112, 112)	1,568	$k=4, s=2, p=1$ ReLU
Conv2	Conv2d	(B, 64, 56, 56)	32,832	$k=4, s=2, p=1$ ReLU
Conv3	Conv2d	(B, 128, 28, 28)	131,200	$k=4, s=2, p=1$ ReLU
Flatten	Reshape	(B, 100352)	0	$128 \times 28 \times 28 = 100,352$
FC → μ	Linear	(B, 128)	~12.8M	Latent mean vector

Layer	Type	Output Shape	Params	Notes
FC → logvar	Linear	(B, 128)	~12.8M	Latent log-variance vector

3.3.2 Reparameterisation Trick

The reparameterisation trick is what makes VAE training possible. To generate a latent vector z for the decoder, we need to sample from the learned Gaussian distribution $N(\mu, \sigma^2)$. However, sampling is a non-differentiable operation — gradients cannot flow through it, so backpropagation would fail.

The trick reformulates the sampling as a deterministic function: $z = \mu + \epsilon \times \exp(0.5 \times \logvar)$, where ϵ is sampled from a standard normal $N(0, I)$. The randomness is now in ϵ (a fixed input), not in the computation graph. Gradients flow freely through μ and $\logvar$, making the model fully trainable end-to-end.

Reparameterisation Formula

$z = \mu + \epsilon \times \sigma$ where $\epsilon \sim N(0, I)$ and $\sigma = \exp(0.5 \times \logvar)$ This separates the stochastic component (ϵ) from the learned parameters ($\mu, \logvar$), enabling gradient backpropagation through the latent sampling step.

3.3.3 Decoder

The decoder receives a sampled latent vector z of dimension 128 and progressively upsamples it back to a full 224×224 RGB image using transposed convolutions (sometimes called deconvolutions). Each transposed convolution doubles the spatial resolution while reducing the channel count.

Layer	Type	Output Shape	Params	Notes
FC projection	Linear	(B, 100352)	~12.8M	$z(128) \rightarrow 128 \times 28 \times 28$
Reshape	View	(B, 128, 28, 28)	0	Restore spatial dims
ConvTranspose1	ConvTranspose2d	(B, 64, 56, 56)	131,136	$k=4, s=2, p=1$ ReLU
ConvTranspose2	ConvTranspose2d	(B, 32, 112, 112)	32,800	$k=4, s=2, p=1$ ReLU
ConvTranspose3	ConvTranspose2d	(B, 3, 224, 224)	1,539	$k=4, s=2, p=1$ Sigmoid
Output	—	(B, 3, 224, 224)	—	Reconstructed image

3.4 Loss Function

The VAE is trained to minimise two objectives simultaneously, combined into a single total loss:

VAE Loss Formula

Total Loss = Reconstruction Loss (MSE) + KL Divergence (KLD)
 $MSE = F.mse_loss(reconstructed_image, original_image, reduction='sum')$
 $KLD = -0.5 \times \sum(1 + \logvar - \mu^2 - \exp(\logvar))$
 MSE pushes the decoder to reproduce the original image accurately. KLD regularises the latent space, pulling the learned distribution toward a standard Gaussian $N(0, I)$. Without KLD the model collapses to a standard deterministic autoencoder and loses the ability to generate new samples.

3.5 Training Metrics — 5 Epoch Log

Epoch	Total Loss	MSE (Reconstruction)	KLD (Latent Reg.)	Status
1	4,520.12	4,410.50	109.62	Warming up
2	3,210.45	3,120.20	90.25	Converging
3	2,540.10	2,465.10	75.00	Stable
4	2,110.85	2,042.30	68.55	Fine-tuning
5	1,845.30	1,780.10	65.20	Converged ☒

The steady decline in both MSE and KLD across all five epochs shows healthy, stable training. The MSE drop from 4,410.50 to 1,780.10 (a 59.6% reduction) indicates the decoder is progressively learning to reconstruct leaf images more accurately. The KLD drop from 109.62 to 65.20 shows the latent space is becoming increasingly structured and Gaussian-like, which is essential for smooth, controllable sample generation.

3.6 Generative Outputs and Latent Space

After training, the VAE generates 16 synthetic leaf samples by sampling from the learned latent space. The generated images are crisp and realistic, showing identifiable disease patterns (yellowing, lesion textures, vein structures) that are visually coherent with real cotton leaf photographs. These synthetic images are used to augment the Fusarium Wilt class in the classifier training set.

PCA Latent Space Visualisation: By projecting the 128-dimensional latent vectors onto 2 principal components, we can visualise how the VAE has organised the data. The PCA plot shows highly distinct, well-separated clusters corresponding to Bacterial Blight, Curl Virus, and Healthy classes. The cluster boundaries are smooth and continuous — confirming that the latent space is well-structured and that interpolating between two points in the space produces biologically meaningful intermediate images.

4. Hyperparameter Search Setup

4.1 Why Principled HPO Matters

In Phase 3, hyperparameters were chosen through manual experimentation — a process that is time-consuming, unreliable, and produces results that cannot be reproduced or explained. Phase 4 replaces this with Optuna, a systematic Bayesian hyperparameter optimisation framework that treats hyperparameter selection as a mathematical search problem.

The key insight behind Bayesian HPO is that trials are not independent. Each trial's result informs the selection of hyperparameters for the next trial — the search algorithm builds a probabilistic model of which hyperparameter regions are likely to perform well and allocates more trials there. This is far more compute-efficient than grid search (which samples uniformly) or random search (which samples without any learning).

4.2 Search Infrastructure

Component	Details
HPO Library	Optuna (open-source Bayesian optimisation framework)
Search Strategy	Tree-Structured Parzen Estimator (TPE) — Bayesian optimisation algorithm
Study Persistence	SQLite database (local file: cottonguard_hpo.db) — study can be paused/resumed
Trial Budget	10 completed trials in this phase
Early Stopping	MedianPruner — kills under-performing trials before completion
Objective Metric	Validation accuracy (maximised) — test set never touched
Reproducibility	All trials seeded; seed fixed per trial for deterministic evaluation
Config File	configs/search_space.yaml — all search bounds defined in one place

4.3 Search Space Definition

We restricted the search space to two hyperparameters. This is an intentional choice — with only 10 trials, spreading the budget across 6–8 dimensions would leave each dimension poorly sampled. Focussing on the two most impactful hyperparameters (learning rate and weight decay, as identified from Phase 3 experience) ensures meaningful coverage.

Hyperparameter	Search Range	Scale	Rationale
Learning Rate (lr)	1×10^{-5} to 1×10^{-2}	Log-scale	LR has an exponential effect on training dynamics; log-scale ensures equal probability mass at each order of magnitude
Weight Decay (wd)	1×10^{-6} to 1×10^{-3}	Log-scale	Controls L2 regularisation strength; log-scale prevents search from over-concentrating on large values

4.4 Search Strategy — Bayesian TPE

The Tree-Structured Parzen Estimator (TPE) algorithm works by maintaining two models of the hyperparameter space: one for high-performing configurations ($l(x)$) and one for low-performing configurations ($g(x)$). At each step, TPE samples candidates that maximise the ratio $l(x)/g(x)$ — in other words, it preferentially samples in regions where good results have been observed and avoids regions that consistently produce poor results.

This strategy converges significantly faster than random search for the same compute budget. In practice, TPE often finds near-optimal configurations in 10–30 trials for problems with 2–4 hyperparameters.

4.5 Early Stopping with MedianPruner

The MedianPruner monitors validation accuracy at the end of each training epoch. If a trial's accuracy falls below the median of all completed trials at the same epoch, the trial is terminated early and the budget is reallocated to more promising configurations. This effectively prevents wasted compute on trials that are clearly under-performing, allowing more trials to be run within the same GPU-hour budget.

5. HPO Results & Analysis

5.1 Trial Log — All 10 Trials

Trial	Learning Rate	Weight Decay	Val Accuracy	Notes
#0	0.000450	1.20×10^{-5}	82.40%	—
#1	0.002100	5.60×10^{-6}	89.10%	—
#2	0.000015	8.90×10^{-5}	74.10%	LR too low; underfitting
#3	0.000850	3.20×10^{-5}	91.25%	—
#4	0.005400	1.80×10^{-4}	86.50%	LR too high; unstable
#5	0.001200	4.50×10^{-5}	94.12%	★ BEST TRIAL
#6	0.008900	2.30×10^{-6}	83.20%	LR too high; overshoot
#7	0.001500	1.20×10^{-5}	92.40%	Close to best
#8	0.000045	6.20×10^{-5}	78.45%	LR too low; slow convergence
#9	0.001100	3.80×10^{-5}	93.80%	Near-optimal

5.2 Selected Best Configuration

Parameter	Value	Outcome
Learning Rate	0.00120	Optimal step size — fast enough to converge, small enough to be stable
Weight Decay	4.50×10^{-5}	Light L2 regularisation — prevents overfitting without over-constraining capacity
Validation Accuracy	94.12%	Best trial; confirmed reproducible across 3 seeds

5.3 Hyperparameter Importance Analysis

Optuna's hyperparameter importance analysis reveals how much each hyperparameter contributed to the variance in validation accuracy across all 10 trials:

Hyperparameter	Importance	Interpretation
Learning Rate	84.2%	Dominant factor. Trials with LR outside the range [0.0008, 0.002] performed significantly worse regardless of weight decay.
Weight Decay	15.8%	Secondary role. Acts as a consistency regulariser — helps best configurations generalise but does not determine the accuracy ceiling.

The dominance of learning rate (84.2% of variance) is a well-known empirical finding in deep learning. The learning rate determines how aggressively the model updates its weights at each step — too large and updates overshoot the loss minimum (causing instability); too small and convergence is too slow for the training budget. The sweet spot for our ResNet-18 on this dataset lies in the narrow band around 0.001–0.002, which the TPE algorithm successfully identified by trial 5.

Weight decay's secondary contribution (15.8%) reflects its role as a consistency regulariser rather than a performance driver. All high-accuracy trials fell in the range $[1e-5, 1e-4]$ for weight decay; extreme values in either direction degraded results moderately but not catastrophically.

5.4 Optimisation History

Plotting validation accuracy against trial number reveals the TPE algorithm's learning behaviour. Trials 0–2 explore different regions of the hyperparameter space, establishing baseline performance across a wide range. By Trial 3, the algorithm has begun concentrating on the mid-range learning rate region (0.0005–0.002). Trials 5, 7, and 9 all cluster in this region and all achieve accuracy above 92%, confirming that the search has converged. The best configuration (Trial 5) was found early, with subsequent trials confirming the robustness of the selected region.

6. Integrated System

6.1 System Summary

 Integrated System Description

Our final integrated model uses a ResNet-18 backbone pretrained on ImageNet-1K, with differential learning rates (1×10^{-5} on shallow layers, 1×10^{-3} on the FC head) selected by Optuna Bayesian search over 10 trials (best config: lr=0.00120, wd= 4.5×10^{-5}). Minority class imbalance — specifically Fusarium Wilt under-representation — is resolved by augmenting the training set with synthetic samples generated by our custom Variational Autoencoder (latent dim=128, trained to MSE=1,780.10, KLD=65.20). The combined system achieves a validation accuracy of 96.20%, representing a +2.07 percentage point improvement over the Phase 3 MobileNetV3-Small result.

6.2 End-to-End Data Flow

The integrated pipeline processes each training batch through four sequential stages:

7. Dataset Preparation: Real cotton leaf images are loaded and split 70/15/15 (train/val/test) using stratified sampling. The training set is augmented with synthetic Fusarium Wilt images generated by the VAE, balancing the minority class.
8. Preprocessing: All images (real and synthetic) pass through the standard pipeline — resize to 224×224, optional CLAHE for low-brightness inputs, and per-channel z-score normalisation using training-set statistics (mean=[0.551, 0.604, 0.521], std=[0.260, 0.244, 0.318]).
9. Feature Extraction and Classification: The preprocessed image is passed through the ResNet-18 backbone (with differential learning rates applied per layer group). The FC head maps the 512-dimensional backbone output to 5 class logits (one per disease category).
10. Loss and Optimisation: Weighted cross-entropy loss (class weights from compute_class_weight) with label smoothing (0.1) is computed. AdamW updates backbone and head parameters at their respective layer-wise learning rates. CosineAnnealingLR modulates all rates toward zero over the training schedule.

6.3 Component Contribution Summary

Component	Problem Solved	Metric Impact	Phase
ResNet-18 Backbone	Richer spatial features than MobileNetV3	+2.07% val acc	Phase 4
Differential LR	Catastrophic forgetting prevention	+1.70% vs full FT	Phase 4
VAE Synthetic Aug.	Fusarium Wilt class imbalance	Tail recall ↑	Phase 4
Optuna HPO (10 trials)	Optimal LR / weight decay selection	94.12% best trial	Phase 4
Phase 3 CNN Baseline	Established 94.13% performance floor	+9.66% vs MLP	Phase 3

7. Negative Results — What Did Not Work

Documenting failures is as important as documenting successes. Two significant training pathologies emerged during Phase 4. Understanding exactly what went wrong — and how we fixed it — is valuable knowledge that informs both our own Phase 5 and future practitioners working on similar problems.

7.1 Catastrophic Forgetting in Full Fine-Tuning

What Happened

When we first attempted Full Fine-Tuning (Configuration 2), we used a uniform learning rate of $1e-3$ across all ResNet-18 layers — a common default for training deep networks from scratch. The result was catastrophic: validation accuracy collapsed to 71.2% after just a few epochs, significantly worse than even Feature Extraction (86.40%).

Diagnosis

This is a textbook catastrophic forgetting event. The shallow layers of ResNet-18 (layers 1 and 2) encode low-level visual features such as edge detectors, colour gradients, and simple textures. These features were learned from 1.28 million ImageNet images and represent years of implicit optimisation. When updated with a large learning rate ($1e-3$), the gradient updates from our small cotton leaf dataset (which has a very different pixel distribution) corrupted these filters within a few batches, destroying the representations that made the pretrained backbone valuable in the first place.

In effect, we wiped out the benefit of pretraining entirely. The model was now trying to learn from scratch, but on far too little data — hence the accuracy collapse.

Catastrophic Forgetting Diagnosis

Symptom: Validation accuracy during full fine-tuning (uniform LR= $1e-3$) dropped to 71.2% — lower than feature extraction (86.40%) and even the Phase 2 MLP baseline (84.47%). Root Cause: High uniform learning rate destroyed ImageNet-pretrained filter weights in shallow backbone layers during early training, eliminating the benefit of pretraining. Fix Applied: Differential Learning Rates — shallow layers updated at $1e-5$ (100× slower), protecting their representations while allowing the deeper layers and head to adapt aggressively.

7.2 Posterior Collapse in VAE Training

What Happened

During initial VAE training experiments, we observed that the KL divergence term in the loss function rapidly collapsed toward zero within the first two epochs. When KLD approaches zero, the encoder outputs an approximately constant distribution ($\mu \approx 0$, $\sigma \approx 1$) regardless of the input image. The decoder then learns to ignore the latent variable z entirely and instead generates a single average image for all inputs. The result: every generated sample looks like a blurry, class-averaged leaf — the model has lost its generative diversity.

Diagnosis

This is known as posterior collapse (or the VAE KLD-vanishing problem). It occurs when the decoder is powerful enough to reconstruct images without using the information in z . The optimiser finds it easier to minimise the total loss by zeroing out the KLD term (making the encoder trivial) and having the decoder memorise average outputs rather than learning a structured latent representation.

Posterior Collapse Diagnosis

Symptom: KLD $\rightarrow 0$ within first 2 epochs. Decoder ignores latent variable z . All generated samples converge to a blurry mean image regardless of the sampled z vector. Root Cause: The decoder was powerful enough to reconstruct adequately without using z . The optimiser found it easier to trivially satisfy KLD by making the encoder output $N(0, I)$ for all inputs. Fix Applied: KL Annealing — the KLD weight β was linearly increased from 0.0 to 1.0 over the first 3 epochs. This allowed the decoder to first learn to reconstruct (using z freely), then gradually forced the encoder to structure the latent space under increasing KLD pressure.

Fix: KL Annealing

KL annealing works by scheduling the weight (β) applied to the KLD term. In epoch 1, $\beta=0$, so the model trains purely as a standard autoencoder and the decoder learns to use z . As β increases linearly to 1.0 over epochs 1–3, the encoder is progressively pressured to structure its output as a proper Gaussian distribution while the decoder has already learned to rely on z for reconstruction quality. This prevents the decoder from bypassing z before it has learned to use it.

8. Plan for Phase 5

8.1 Phase 5 Objective

Phase 5 is the final evaluation and error analysis phase. Having built, refined, and integrated the model in Phases 3 and 4, Phase 5 will produce the definitive, honest assessment of CottonGuard AI's capabilities and limitations. This includes comprehensive per-class metrics, systematic ablation studies, a detailed error analysis, and a final test-set evaluation (the first and only time the test set will be used).

8.2 Evaluation Metrics Plan

Phase 3 and 4 primarily used validation accuracy as the evaluation metric. Accuracy is problematic for imbalanced datasets because a model can achieve high accuracy by correctly classifying the majority class while failing on minority classes. Phase 5 will use a richer suite of metrics:

Metric	What It Measures	Why Important for CottonGuard
Macro-Avg F1 Score	Equal-weighted average F1 across all classes	Penalises failure on rare classes (Fusarium Wilt Critical) equally to failure on common ones
Per-Class Precision	Fraction of positive predictions that are correct	Reveals which disease classes have too many false alarms (false positives)
Per-Class Recall	Fraction of actual positives correctly identified	Critical for diagnosis — a missed disease is more dangerous than a false alarm
Confusion Matrix	Full NxN matrix of predicted vs actual class	Identifies which classes are most commonly confused with each other
ROC-AUC (per class)	Area under receiver operating characteristic curve	Threshold-independent measure of discriminative power for each class
Calibration Error (ECE)	Alignment between predicted confidence and accuracy	Ensures the model's probability outputs are trustworthy for field use

8.3 Ablation Studies Planned

Phase 5 will run controlled ablation experiments to quantify the contribution of each Phase 4 component:

11. VAE Augmentation Ablation: Train the final ResNet-18 model without VAE-generated synthetic images. Compare macro F1 and Fusarium Wilt recall against the full system to isolate the VAE's contribution.
12. Differential LR vs Frozen vs Full FT: Re-run the Phase 5 final evaluation on all three transfer learning configurations to confirm that Differential LR's advantage persists on the test set.
13. HPO Configuration Sensitivity: Evaluate the top 3 Optuna trials (Trials 5, 9, 7) on the test set to confirm that the selected best configuration is genuinely better and not a validation-set artefact.
14. Augmentation Pipeline Ablation: Remove one augmentation type at a time (HFlip, Affine, ColorJitter, CLAHE) to identify which augmentations contribute most to generalisation.

8.4 Error Analysis Plan

A detailed per-example error analysis will be conducted on the test set, following the Phase 3 approach but at greater depth:

- Sort all test set predictions by cross-entropy loss (highest loss = most confidently wrong predictions).
- Examine the top 20 worst predictions and categorise them by failure mode: inter-class visual ambiguity, early-stage disease, abiotic vs biotic confusion, or image quality issues.
- Apply Grad-CAM to each misclassified example to verify whether the model attended to the correct region of the image — distinguishing genuine feature confusion from attention errors.
- Compute confusion matrices at the disease-type level and separately at the severity level, to determine whether Phase 4's improvements reduced inter-disease or intra-disease confusion more.

8.5 Final Test Set Evaluation

The test set will be evaluated exactly once at the end of Phase 5. No hyperparameter decisions or model selection choices will be made after seeing test set results. The final configuration to be evaluated is: ResNet-18 backbone with Differential LR (layer rates $1e-5$ / $1e-4$ / $1e-3$), HPO-selected hyperparameters ($lr=0.00120$, $wd=4.5e-5$), and training set augmented with VAE-generated Fusarium Wilt synthetic images.

Phase 5 Success Criteria

Primary: Macro-averaged F1 > 0.93 across all five disease classes on the held-out test set. Secondary: Fusarium Wilt class recall > 0.88 (the class most damaged by imbalance in Phase 3). Tertiary: Calibration Error (ECE) < 0.05, ensuring the model's confidence scores are meaningful for real-world deployment decisions.